

Grammarcetamol — Admin Frontend Architecture & Design Specification

Document Version: 1.0
Application: Grammarcetamol-Admin-Interface
Stack: Next.js (App Router), TypeScript, Tailwind CSS, React Query, Zustand, Recharts
Status: Draft
Last Updated: 2026-07-31

1. Overview

The **Admin Interface** is the operational control center of the Grammarcetamol ecosystem. Unlike the student-facing application, which prioritizes learning and engagement, the admin portal is engineered for **efficiency, clarity, and control**. Every screen is designed to help administrators manage educational content, monitor business performance, handle student interactions, and maintain platform integrity — all from a single, cohesive interface.

The admin application serves two distinct roles with tiered permissions, ensuring that sensitive operations remain restricted while empowering trusted assistants to contribute meaningfully.

1.1 Design Philosophy

- **Data density over whitespace:** Administrators need to see more information per screen.
- **Action-oriented:** Every view should surface the next logical action.
- **Consistency across modules:** Uniform patterns for tables, forms, and workflows reduce training time.
- **Performance at scale:** Tables must handle thousands of rows with virtualization; charts must render smoothly with large datasets.
- **Reliability indicators:** System health, upload status, and error states must be immediately visible.

1.2 Target Users

Role	Permissions	Primary Responsibilities
Super Admin	Unrestricted	Full platform ownership: courses, students, revenue, configuration, security
Moderator	Configurable	Content publishing, support responses, class scheduling, review moderation (no financial/settings access)

2. Information Architecture

```
grammarcetamol-admin-interface/  
├─ Authentication
```

├─ /login	→ Admin Login (Email + Password, 2FA optional)
└─ /forgot-password	→ Password Reset
├─ Dashboard	
└─ /	→ Operational Control Center (Metrics, Quick Actions, Activity Feed)
├─ Course Management	
├─ /courses	→ Course List (All states: Draft, Uploading, Review, Published, Archived)
├─ /courses/create	→ Step-by-Step Course Creation Wizard
├─ /courses/[id]	→ Course Detail & Editor
├─ /courses/[id]/upload	→ Chunked Upload Manager
├─ /courses/[id]/analytics	→ Per-Course Engagement Analytics
└─ /courses/[id]/versions	→ Course Version History & Rollback
├─ Live Classroom Management	
├─ /live-classes	→ Scheduled Classes Calendar & List
├─ /live-classes/create	→ New Live Class Scheduler
├─ /live-classes/[id]	→ Class Detail & Attendance
└─ /live-classes/[id]/edit	→ Edit Scheduled Class
├─ Student Management	
├─ /students	→ Student Directory (Search, Filter, Export)
├─ /students/[id]	→ Student Profile & Activity Timeline
├─ /students/[id]/progress	→ Individual Learning Progress
└─ /students/[id]/transactions	→ Student Payment History
├─ Instructor Management (Future)	
├─ /instructors	→ Instructor Directory
└─ /instructors/[id]	→ Instructor Profile & Availability
├─ Service Requests	
├─ /service-requests	→ Incoming Requests (Filterable Pipeline)
└─ /service-requests/[id]	→ Request Detail & Response Workflow
├─ Reviews & Feedback	
├─ /reviews	→ All Reviews (Pending, Approved, Flagged)
└─ /reviews/[id]	→ Review Detail & Moderation Actions
├─ Financial Management	
├─ /revenue	→ Revenue Dashboard (Charts, Reports)
├─ /transactions	→ All Transactions (Payments, Refunds, Payouts)
├─ /payouts	→ Instructor/Partner Payouts (Future)
└─ /invoices	→ Invoice Generation & History
├─ Analytics & Reports	
├─ /analytics/overview	→ Platform-Wide Engagement Metrics
├─ /analytics/students	→ Student Behavior & Retention
├─ /analytics/courses	→ Course Performance & Completion Rates
├─ /analytics/revenue	→ Revenue Trends & Forecasting
└─ /analytics/search	→ Search Keyword Analytics

└─ Notifications & Communication		
└─ /announcements	→ Platform Announcement Manager	
└─ /announcements/create	→ Compose & Schedule Announcements	
└─ /notifications	→ Notification Templates & History	
└─ /email-templates	→ Transactional Email Editor	
└─ Support & Tickets		
└─ /support/tickets	→ Support Ticket Queue	
└─ /support/tickets/[id]	→ Ticket Detail & Threaded Response	
└─ /support/knowledge-base	→ FAQ & Help Article Manager	
└─ User & Role Management		
└─ /users	→ All User Accounts (Admins, Moderators)	
└─ /users/create	→ Create New Admin/Moderator	
└─ /users/[id]	→ User Detail & Permission Editor	
└─ /roles	→ Role & Permission Configuration	
└─ System & Settings		
└─ /settings/general	→ Platform Configuration (Name, Logo, Contact)	
└─ /settings/payments	→ Payment Gateway Configuration	
└─ /settings/notifications	→ Notification Channel Settings	
└─ /settings/security	→ Security Policies (Password rules, 2FA, Session)	
└─ /settings/integrations	→ Third-Party Service Integrations	
└─ /settings/backup	→ Data Export & Backup	
└─ Logs & Audit		
└─ /logs/system	→ System Logs & Error Tracking	
└─ /logs/audit	→ Admin Action Audit Trail	
└─ /logs/uploads	→ Upload Session Logs	

3. Global Design System

3.1 Color Palette

The admin interface uses a more subdued, professional palette than the student app, emphasizing information hierarchy and reducing eye strain during long operational sessions.

Token	Hex	Usage
--color-primary	#1E3A5F	Primary brand, navigation, key actions
--color-primary-light	#2A5285	Hover states, secondary buttons
--color-accent	#0EA5E9	Information highlights, links, active states
--color-accent-hover	#0284C7	Link hover, active tab underline
--color-success	#10B981	Success, published, completed, active
--color-success-light	#D1FAE5	Success background tint

Table 2 – continued

Token	Hex	Usage
--color-warning	#F59E0B	Warnings, pending, uploading, needs attention
--color-warning-light	#FEF3C7	Warning background tint
--color-error	#EF4444	Errors, failed, rejected, blocked
--color-error-light	#FEE2E2	Error background tint
--color-info	#6366F1	Informational badges, neutral highlights
--color-info-light	#E0E7FF	Info background tint
--color-background	#F1F5F9	Page backgrounds
--color-surface	FFFFFF	Cards, panels, tables, modals
--color-surface-hover	#F8FAFC	Table row hover, selectable item hover
--color-border	#E2E8F0	Dividers, table borders, input borders
--color-border-strong	#CBD5E1	Focused borders, active dividers
--color-text-primary	#0F172A	Headings, primary text
--color-text-secondary	#475569	Body text, descriptions
--color-text-muted	#94A3B8	Placeholders, disabled, timestamps
--color-sidebar	#0F172A	Sidebar background
--color-sidebar-text	#CBD5E1	Sidebar text
--color-sidebar-active	#1E3A5F	Sidebar active item background

3.2 Typography

Element	Font	Weight	Size	Line Height
H1 (Page Title)	Inter	700	28px	1.2
H2 (Section)	Inter	600	22px	1.3
H3 (Card Title)	Inter	600	18px	1.4
H4 (Subsection)	Inter	600	16px	1.4
Body	Inter	400	14px	1.5
Body Small	Inter	400	13px	1.4
Caption / Label	Inter	500	12px	1.3
Data / Metric	Inter	700	32px	1.0
Table Header	Inter	600	12px	1.3
Table Cell	Inter	400	13px	1.4
Button	Inter	600	14px	1.0
Nav Link	Inter	500	14px	1.0

3.3 Spacing Scale

Based on a 4px grid (tighter than student app for data density):

Token	Value
space-1	4px
space-2	8px
space-3	12px
space-4	16px
space-5	20px
space-6	24px
space-7	32px
space-8	40px
space-9	48px
space-10	64px

3.4 Border Radius

Token	Value	Usage
radius-sm	4px	Buttons, badges, inputs
radius-md	6px	Cards, table cells, small panels
radius-lg	8px	Modals, large cards, dropdowns
radius-xl	12px	Feature panels, charts container

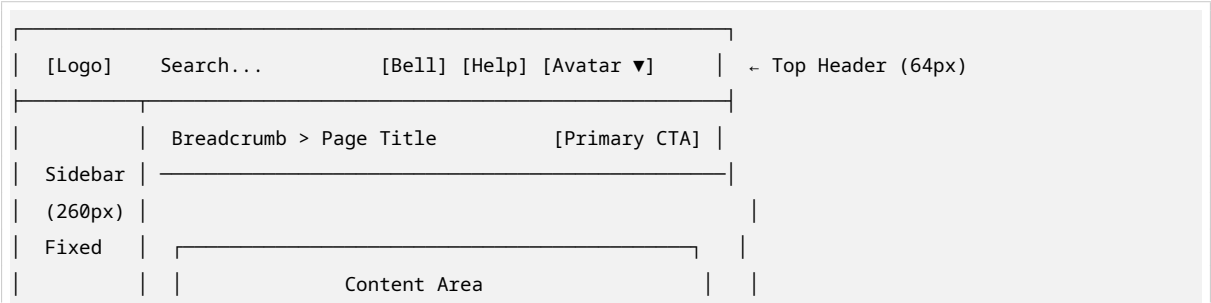
3.5 Shadows

Token	Value	Usage
shadow-sm	0 1px 2px rgba(0,0,0,0.05)	Subtle elevation, table rows
shadow-md	0 4px 6px -1px rgba(0,0,0,0.08)	Cards, dropdowns, popovers
shadow-lg	0 10px 15px -3px rgba(0,0,0,0.08)	Modals, drawers, toasts
shadow-xl	0 20px 25px -5px rgba(0,0,0,0.1)	Overlays, confirmation dialogs

4. Layout Architecture

4.1 Shell Layout

The admin interface uses a **persistent sidebar + top header + content area** layout pattern, optimized for extended use and rapid navigation between modules.



Nav		(Fluid width, max-width 1440px centered)	
Menu			
		Cards Tables Forms Charts Tabs	

4.2 Sidebar Navigation

Structure:

- **Header:** Collapsed logo icon (40px) + Expanded logo text
- **Navigation Groups:**
- **Overview:** Dashboard
- **Education:** Courses, Live Classes, Reviews
- **People:** Students, Instructors (future), Users & Roles
- **Business:** Service Requests, Revenue, Transactions
- **Insights:** Analytics, Search Analytics
- **Communication:** Announcements, Notifications, Support Tickets
- **System:** Settings, Logs & Audit
- **Each Item:** Icon (20px) + Label + Badge (if notifications) + Expand chevron (for groups)
- **Active State:** `--color-sidebar-active` background, left border accent (3px), white text
- **Hover State:** Slight background lighten
- **Collapse:** Pin/unpin toggle at bottom; fully collapsible to icon-only mode (72px)

Responsive Behavior:

- Desktop (lg+): Persistent sidebar
- Tablet (md): Collapsible overlay drawer
- Mobile (< md): Hidden by default, hamburger toggle reveals full-screen overlay

4.3 Top Header

Height: 64px

Background: `--color-surface` with bottom border

Elements (left to right):

1. **Sidebar Toggle:** Hamburger icon (mobile/tablet only)
2. **Search Bar:** Global search with command palette (Cmd+K shortcut)
 - Searches: Courses, Students, Transactions, Support Tickets
 - Results grouped by category with keyboard navigation
3. **Right Cluster:**

- **Notification Bell:** Dropdown with recent admin alerts (upload failures, new registrations, failed payments)
- **Help Icon:** Links to documentation or opens help modal
- **User Avatar:** Dropdown with Profile, Settings, Logout

4.4 Breadcrumb & Page Header

Breadcrumb:

- Format: Dashboard > Module > Page > Action
- Each segment is a link except the current page
- Truncates middle segments on narrow viewports

Page Header:

- Left: Page title (H1) + Optional subtitle/description
- Right: Primary action button(s) + Secondary actions (export, refresh, filter toggle)
- Sticky on scroll (optional, for long pages)

5. Component Inventory

5.1 Data Display Components

DataTable The cornerstone of the admin interface. Used across Courses, Students, Transactions, Reviews, and Tickets.

Features:

- Column definitions: Header, accessor, sortable, filterable, width, cell renderer
- Row selection: Checkbox per row + header checkbox (select all)
- Bulk actions: Appear in floating bar when rows selected (Delete, Export, Change Status)
- Pagination: Page size selector (10, 25, 50, 100) + Page navigation + “Showing X–Y of Z results”
- Sorting: Click header to sort asc/desc; multi-column sort with Shift+click
- Filtering: Column-specific filters (text, date range, dropdown, boolean)
- Empty state: Centered illustration + message + CTA
- Loading state: Skeleton rows (5 rows)
- Row actions: Dropdown menu (View, Edit, Delete) or inline icon buttons
- Expandable rows: Click to reveal detail panel below row
- Sticky header: Header remains visible on vertical scroll
- Horizontal scroll: On overflow, with shadow indicators

Variants:

- **Compact:** Reduced padding for dense data views

- **Comfortable:** Default padding
- **Card-contained:** Table inside a card with title and actions

StatCard (Dashboard Metric)

- **Layout:** Icon (40px, rounded, colored background) + Label (caption, muted) + Value (H2, bold) + Change indicator (arrow + percentage + period)
- **Trend:** Sparkline mini-chart below value (optional, 7-day trend)
- **Clickable:** Entire card is a link to detailed report
- **States:** Loading (skeleton), Error (retry button)

StatusBadge

- **Sizes:** Small (for tables), Medium (for cards), Large (for detail headers)
- **Variants:**
 - success: Published, Active, Completed, Paid
 - warning: Draft, Uploading, Pending, Processing
 - error: Failed, Rejected, Blocked, Expired
 - info: Review, Scheduled, In Progress
 - neutral: Archived, Cancelled, Refunded
- **Style:** Pill shape, colored background tint, colored text, optional dot indicator

ProgressBar

- **Usage:** Upload progress, course completion, task completion
- **Specs:** Height 8px, rounded, track `--color-border`, fill `--color-accent` or `--color-success`
- **Label:** Percentage text to the right (optional)
- **Animated:** Smooth width transition on progress change

UploadQueueItem

- **Usage:** Individual file row within the FileUploader queue
- **Structure:**
 - Left: File type icon (video/document) + File name (truncated with tooltip) + File size
 - Center: Lesson assignment dropdown (“Assign to Lesson...” → lists unassigned lessons) + Progress bar + Status text
 - Right: Speed/ETA (when uploading) + Action buttons (Pause/Resume, Retry, Cancel, Remove)
- **Expandable:** Click to expand and show per-chunk status grid
 - Chunk grid: 50 small squares representing chunks, colored by status (gray=pending, blue=uploading, green=done, red=failed)
 - Technical details: Chunk size, checksum, retry count
- **Context menu:** Right-click for additional options (Reassign Lesson, Move to Top, Move to Bottom, Copy Error Message)

ActivityFeedItem

- **Structure:** Icon (colored by type) + Actor + Action + Target + Timestamp + Link
- **Types:** Registration (user icon), Purchase (dollar icon), Upload (cloud icon), Error (alert icon), Review (star icon), Ticket (message icon)

- **Grouping:** Consecutive items by same actor collapsed with “+3 more” expander
- **Real-time:** New items slide in from top with subtle highlight fade

5.2 Form Components

FormSection

- **Structure:** Section title (H3) + Optional description + Divider + Form fields grid
- **Grid:** 1 column (mobile), 2 columns (desktop default), configurable

RichTextEditor

- **Usage:** Course descriptions, announcement bodies, email templates
- **Toolbar:** Bold, Italic, Lists, Links, Headings, Image embed, Code block
- **Output:** HTML or Markdown
- **Validation:** Max length counter, required indicator

ImageUploader

- **Drag & Drop zone:** Dashed border, cloud upload icon, “Click or drag files here”
- **Preview:** Thumbnail with remove button, file name, size
- **Validation:** Max size, accepted formats, dimension constraints
- **Cropper:** Optional image crop before upload (for avatars, thumbnails)

FileUploader (Chunked — Course Content)

- **Purpose:** Resumable, fault-tolerant upload of large video and resource files for course lessons
- **Drop Zone:** Large dashed-border area with icon + “Drag video files here or click to browse” + Supported formats hint (MP4, MOV, MKV, PDF, etc.)
- **File Queue Panel:**
 - Each file row: File icon | Name | Size | Assigned Lesson (dropdown) | Status | Progress | Speed | ETA | Actions
 - **Status states:**
 - * 🕒 **Waiting** (gray): Queued, not yet started
 - * 🕒 **Uploading** (blue): Animated progress bar + percentage + speed (e.g., “2.4 MB/s”) + ETA (e.g., “~3 min remaining”)
 - * ⏸ **Paused** (yellow): Progress frozen, resume button available
 - * ✅ **Uploaded** (green): Checkmark + “Processing...” → “Ready” once backend confirms
 - * ❌ **Failed** (red): Error message (e.g., “Network timeout”, “Invalid format”) + Retry button + “Skip” option
 - * 🚫 **Cancelled** (muted): Strikethrough, removable from queue
 - **Progress bar:** Segmented to show per-chunk completion; overall percentage to the right
- **Global Controls (toolbar above queue):**
 - “Add More Files” button
 - “Pause All” / “Resume All” (toggles based on active uploads)
 - “Clear Completed” (removes successful uploads from view)
 - “Cancel All” (aborts all active and pending uploads)
 - Network status indicator (Online / Offline / Slow)
- **Session Recovery Banner:**
 - Appears on page load if an interrupted session is detected

- “You have an unfinished upload for ‘[Course Name]’. Resume where you left off?”
- Actions: “Resume Upload” | “Start Fresh” (abandons previous session)
- Shows: Session start time, files completed / total, last activity
- **Aggregate Progress:**
 - Large progress ring or bar at top: “X of Y files uploaded (Z%)”
 - Estimated total time remaining
 - Total data transferred / total size
- **Validation (pre-upload):**
 - File type whitelist: MP4, MOV, AVI, MKV, WEBM (video); PDF, DOCX, PPTX (resources)
 - Max file size: 10GB per video, 100MB per resource
 - Min video duration: 30 seconds (warn if below)
 - Duplicate detection: Warn if filename already exists in course
- **Drag & Drop Reordering:** Reorder files in queue to change upload priority

VideoUploader (Specialized)

- **Extends:** FileUploader with video-specific features
- **Preview:** Thumbnail extraction from first frame (auto-generated or custom upload)
- **Metadata display:** Duration, Resolution, Codec, Frame rate, Bitrate (extracted client-side via fprobe-wasm or post-upload)
- **Transcoding status:** If backend transcodes to multiple resolutions (720p, 1080p), show per-resolution progress
- **Poster frame selector:** Scrubber to pick thumbnail frame from video

DateTimePicker

- **Usage:** Live class scheduling, announcement publishing
- **Components:** Date picker (calendar popup) + Time picker (dropdown) + Timezone selector
- **Validation:** Prevent past dates, conflict detection

MultiSelect

- **Usage:** Categories, tags, permissions, target audiences
- **Interaction:** Type to search + dropdown checklist + selected pills (removable)
- **Bulk:** Select all / Clear all options

5.3 Feedback Components

Toast (Admin Variant)

- **Position:** Top-right (desktop), top-center (mobile)
- **Types:** Success, Error, Warning, Info
- **Duration:** 5 seconds (auto-dismiss), persistent for errors requiring action
- **Actions:** Undo button (for destructive actions), View Details link

ConfirmationDialog

- **Usage:** Delete confirmations, status changes, bulk actions
- **Structure:** Warning icon + Title + Description + Consequence list + Cancel + Confirm (destructive styling)
- **Destructive:** Confirm button uses `--color-error` background

- **Requires typing:** For high-risk actions (e.g., “Type DELETE to confirm”)

EmptyState

- **Variants:**
 - **Search:** Magnifying glass + “No results found” + Clear filters
 - **Create:** Plus icon + “No [items] yet” + Create button
 - **Error:** Alert triangle + “Failed to load” + Retry
 - **Success:** Checkmark + “All caught up!”

5.4 Chart Components (Recharts)

LineChart

- **Usage:** Revenue trends, student growth, active users over time
- **Features:** Multiple series, tooltip on hover, legend toggle, zoom brush, data points

BarChart

- **Usage:** Course sales comparison, category performance, monthly transactions
- **Features:** Grouped bars, stacked bars, horizontal variant, value labels on bars

PieChart / DonutChart

- **Usage:** Revenue by category, enrollment distribution, payment method split
- **Features:** Percentage labels, legend, center text (total), interactive slices

AreaChart

- **Usage:** Cumulative metrics, traffic over time
- **Features:** Gradient fill, stacked areas, smooth curves

DataTableChart (Hybrid)

- **Usage:** Tables with embedded sparklines (student activity, course engagement)
- **Features:** Mini line chart in table cell, hover tooltip

6. Page Specifications

6.1 Dashboard (/)

Objective: Provide an at-a-glance operational overview of the entire platform.

Layout: Full-width content, no sidebar sub-navigation.

Sections:

1. Metric Cards Row

- 4-column grid (desktop), 2-column (tablet), 1-column (mobile)
- Cards:
 - **Total Students:** Count + “+X this month” trend + Sparkline
 - **Active Students (30d):** Count + Percentage of total + Trend

- **Total Revenue:** Amount + “+X this month” + Sparkline
- **Pending Actions:** Count of uploads needing review + open tickets + pending requests
- Each card clickable → drills to respective detail page

2. Quick Actions Bar

- Horizontal row of prominent buttons:
 - “+ Create Course” (primary)
 - “Schedule Live Class” (secondary)
 - “Publish Announcement” (secondary)
 - “Add Moderator” (secondary)
 - “Review Uploads” (secondary, with pending count badge)
 - “View Report” (ghost)
- Sticky below metric cards on scroll

3. Two-Column Layout Below

Left Column (65%):

a. Revenue Overview Chart

- Tabs: Daily | Weekly | Monthly
- Line chart: Revenue over selected period
- Overlay: Payment success vs. failure ratio
- “View Full Report” link

b. Student Insights

- Tabs: Most Active | At Risk | New This Week
- Compact data table: Name | Courses Enrolled | Time Spent | Last Active | Action (View Profile)
- “View All Students” link

Right Column (35%):

a. Activity Feed

- Title: “Recent Activity” + “View All Logs” link
- Scrollable list (max 10 items, then “Load more”)
- Real-time updates with subtle animation
- Filter: All | Errors | Sales | Uploads | Registrations

b. Upcoming Live Classes

- Mini list: Time | Title | Instructor | Capacity | Action (Edit/Join)
- “View Calendar” link

c. Pending Service Requests

- Count badge + Latest 3 requests
- Quick status update buttons (“Mark as Reviewed”)

6.2 Course List (/courses)

Layout: Page header + Filter bar + DataTable

Filter Bar:

- Search input (searches title, instructor, tags)
- Status dropdown: All | Draft | Uploading | Processing | Review | Published | Archived

- Category dropdown
- Date range picker (created/updated)
- “Clear Filters” button
- “Export CSV” button (exports filtered results)

DataTable Columns:

- Checkbox (bulk select)
- Thumbnail (40px) + Title (clickable → detail)
- Instructor
- Category
- Status (`StatusBadge`)
- Price
- Students Enrolled
- Rating
- Last Updated
- Actions: View | Edit | Duplicate | Archive/Delete

Bulk Actions Bar (appears on selection):

- “Publish Selected” (if all Draft/Review)
- “Archive Selected”
- “Export Selected”
- “Delete Selected” (only if no enrollments)
- Selection count: “X courses selected”

Empty States:

- No courses: “No courses found. Create your first course.” + CTA
- No filter results: “No courses match your filters.” + Clear CTA

6.3 Course Creation Wizard (/courses/create)

Layout: Step indicator (top, sticky) + Form area + Action footer

Step Indicator:

- Horizontal progress bar with numbered steps:
 1. Course Information
 2. Pricing & Availability
 3. Build Structure
 4. Upload Content
 5. Review & Publish

- Completed steps: Checkmark + green
- Current step: Active highlight + bold
- Future steps: Muted
- Clicking completed step navigates back (with unsaved changes warning)

Step 1: Course Information

- Form fields (2-column grid where applicable):
- Course Title* (text, max 100 chars)
- Subtitle (text, max 200 chars)
- Description* (`RichTextEditor`)
- Learning Objectives* (repeatable text inputs, min 3)
- Target Audience (text area)
- Prerequisites (text area)
- Category* (dropdown)
- Difficulty* (radio: Beginner | Intermediate | Advanced)
- Language* (dropdown, default: English)
- Estimated Duration (number + unit selector)
- Cover Image* (`ImageUploader`, 16:9, min 1280×720)
- Promotional Video (optional, `VideoUploader`)
- Tags (multi-select or comma-separated input)
- Auto-save indicator: “Saved 2 minutes ago” or “Unsaved changes”
- “Save as Draft” button (always available)
- “Next: Pricing” button

Step 2: Pricing & Availability

- Pricing Model: Free toggle
- If Paid: Price input (currency selector + amount), Discount Price (optional), Discount expiry
- If Free: Skip to next step
- Enrollment Period: Start date / End date (optional, for limited enrollments)
- Early-bird toggle (future placeholder)
- “Back” + “Next: Build Structure” buttons

Step 3: Build Structure

- **Module Manager:**
- Module list with drag-and-drop reordering
- Each module: Title (inline editable) + Collapse/Expand + Actions (menu: Rename, Duplicate, Delete, Add Lesson)

- “+ Add Module” button (bottom)
- **Lesson List (inside expanded module):**
- Drag-and-drop lesson reordering
- Each lesson: Drag handle + Title (inline editable) + Type icon (Video, Text, Quiz) + Duration + Actions (menu: Edit, Duplicate, Move to Module, Delete)
- “+ Add Lesson” button (per module)
- **Lesson Detail Panel (right side or modal):**
- Lesson Title
- Lesson Type: Video | Text | Resource
- Description
- Attachments list
- “Save Lesson” button
- **Structure Preview:** Collapsible tree view of entire course
- “Back” + “Next: Upload Content” buttons

Step 4: Upload Content

- **Upload Manager Interface:**
- File drop zone (supports multiple files)
- Lesson-to-file mapping: Each lesson shows assigned file or “Assign File” button
- Per-file progress: Chunked upload with resume capability
- Overall progress: Aggregate bar + “X of Y files uploaded”
- File status indicators:
- Uploaded (green)
- 🕒 Uploading (animated progress bar + speed + ETA)
- Paused (yellow, resume button)
- Failed (red, retry button + error message)
- ⌛ Waiting (gray)
- **Upload Session Panel:**
- Session ID, start time, estimated completion
- “Pause All” / “Resume All” / “Cancel All” controls
- Network status indicator
- Auto-resume on browser re-open: Detects existing upload session, prompts to resume
- “Back” + “Next: Review” buttons

Step 5: Review & Publish

- **Validation Checklist:**

- All required fields completed
- Cover image uploaded
- At least one module with one lesson
- All lessons have content assigned
- Pricing configured
- No duplicate lesson titles
- Video files validated (format, duration)
- **Preview Mode:** Toggle to see exactly how the course appears to students
- **Publish Options:**
 - “Publish Now” (immediate)
 - “Schedule Publish” (date/time picker)
 - “Save as Draft” (return to editing)
- **Pre-publish Warning:** “This course will be visible to students. Are you sure?”
- Post-publish: Success toast + “View Course” link + “Create Another Course” button

6.4 Course Editor (/courses/[id])

Layout: Tabbed interface for managing a published or draft course.

Tabs:

1. **Overview:** Read-only summary + key metrics (enrollments, revenue, rating)
2. **Edit:** Same form structure as creation wizard, but with change tracking
 - Highlight modified fields
 - “Unsaved changes” warning on navigate away
 - “Save Changes” + “Discard Changes” buttons
3. **Content:** Module/lesson tree with inline editing
 - Add new lessons/modules
 - Reorder via drag-and-drop
 - Replace video files
 - Lesson analytics (views, average watch time)
4. **Students:** Enrolled students table
 - Columns: Name | Enrolled Date | Progress | Last Active | Certificate (if completed)
 - Export enrollment list
5. **Analytics:** Per-course charts
 - Enrollment trend (line chart)
 - Revenue (if paid)

- Student engagement (lessons completed, time spent)
- Rating distribution (bar chart)

6. **Versions:** Version history table

- Columns: Version | Date | Author | Changes Summary | Actions (View, Restore)
- “Create New Version” button (manual snapshot)

6.5 Chunked Upload Manager (/courses/[id]/upload)

Dedicated interface for managing large media uploads.

Header: Course name + “Upload Manager” subtitle + Overall progress

Upload Queue Table:

- Columns: File Name | Lesson Assignment | Size | Status | Progress | Speed | ETA | Actions
- **Actions per row:**
 - Pause/Resume
 - Retry (if failed)
 - Cancel
 - Reassign Lesson
- **Global Controls:**
 - “Add More Files” button
 - “Pause All” / “Resume All”
 - “Clear Completed”
 - Network status indicator

Session Recovery Panel:

- Detects interrupted sessions on page load
- “Resume Previous Upload” prompt with session details
- “Start Fresh” option (abandons previous session)

Technical Display (for debugging):

- Chunk size configuration
- Concurrent upload limit
- Checksum verification status
- Expandable row showing per-chunk status

6.6 Live Class Scheduler (/live-classes & /live-classes/create)

List View (/live-classes):

- **View Toggle:** Calendar view (month/week/day) | List view
- **Filter Bar:** Date range, Instructor, Status (Upcoming | Live | Completed | Cancelled), Capacity

- **List Table Columns:**

- Title | Date & Time | Duration | Instructor | Capacity (Enrolled/Max) | Price | Status | Actions

- **Calendar View:**

- Color-coded events by status

- Click event → detail sidebar

- Drag to reschedule (with conflict detection)

- **Quick Create:** “+ Schedule Class” button → modal with essential fields

Create/Edit Form:

- Class Title* (text)

- Description (RichTextEditor)

- Instructor* (dropdown, filtered by availability)

- Date* (date picker)

- Start Time* + End Time* (time pickers)

- Timezone (auto-detected, editable)

- Capacity* (number, default 30)

- Pricing: Free toggle | Price input

- Meeting Link (URL) or Platform Selection (Zoom, Jitsi, Google Meet)

- Cover Image (optional)

- **Conflict Detection:** Real-time check against instructor’s other classes + room availability

- **Student Notification:** Checkbox “Notify enrolled students of changes” (on edit)

6.7 Student Directory (/students)

Layout: Page header + Advanced filter sidebar (collapsible) + DataTable

Filter Sidebar:

- Search: Name, Email, Phone

- Registration Date: Date range

- Status: Active | Suspended | Unverified | All

- Enrollment Count: Min / Max

- Last Active: Today | This Week | This Month | Custom Range

- Country: Multi-select dropdown

- Courses Enrolled: Multi-select

DataTable Columns:

- Avatar + Full Name (clickable → profile)

- Email

- Phone
- Country
- Registration Date
- Enrollments Count
- Last Active
- Account Status (`StatusBadge`)
- Actions: View | Edit | Suspend/Activate | Email

Student Profile Page (`/students/[id]`):

- **Header:** Large avatar + Name + Email + Status badge + Actions (Edit, Suspend, Email, Delete)
- **Info Grid:** Registration date | Country | Timezone | Phone | Learning Goals | Bio
- **Tabs:**
- **Activity Timeline:** Chronological events (enrollments, completions, payments, logins)
- **Enrollments:** Course cards with progress bars
- **Progress:** Detailed per-course breakdown (modules completed, time spent)
- **Transactions:** Payment history table
- **Notes:** Internal admin notes (only visible to admins)

6.8 Revenue Dashboard (`/revenue`)

Layout: Full-width charts + Summary cards + Detailed table

Summary Cards (top row):

- Total Revenue (lifetime)
- Revenue This Month (vs. last month)
- Revenue This Week
- Today's Revenue
- Average Transaction Value
- Refund Rate

Chart Section:

- **Main Chart:** Line chart, toggle between Daily / Weekly / Monthly / Yearly
- Series: Gross Revenue | Net Revenue | Refunds
- Date range picker
- Zoom/pan enabled
- **Secondary Charts (2-column grid):**
- Revenue by Course Category (Donut chart)
- Revenue by Payment Method (Bar chart)

- Best-Selling Courses (Horizontal bar chart, top 10)
- Payment Success vs. Failure (Stacked bar, monthly)

Transactions Table (/transactions):

- Columns: Transaction ID | Date | Student | Course/Service | Amount | Payment Method | Status | Actions
- Filters: Date range, Status, Payment method, Amount range
- Export to CSV/Excel
- Refund action (with confirmation and reason input)

6.9 Service Request Pipeline (/service-requests)

Layout: Kanban board view (default) + List view toggle

Kanban Columns:

- **Received** (gray)
- **Under Review** (blue)
- **Responded** (purple)
- **Scheduled** (orange)
- **Completed** (green)
- **Cancelled** (red)

Card Structure:

- Requester name + Organization
- Service type badge
- Submission date
- Priority indicator (dot color)
- Assigned admin avatar
- Quick actions: View | Assign | Change Status

Drag-and-drop: Move cards between columns to update status (with confirmation modal for destructive moves)

Detail View (/service-requests/[id]):

- **Header:** Request ID + Status + Priority + Assigned to + Date
- **Requester Info:** Name, Email, Phone, Organization
- **Service Details:** Type, Description, Preferred Dates, Budget (if provided)
- **Internal Notes:** Threaded admin-only notes
- **Communication Timeline:** All emails sent, status changes, admin responses
- **Actions:** Change Status | Assign to Admin | Send Email | Mark as Spam | Archive

6.10 Support Ticket Queue (/support/tickets)

Layout: Split-pane (ticket list left, detail right)

Left Pane (Ticket List):

- Filter tabs: All | Open | In Progress | Resolved | Closed | Critical
- Search by ticket ID, student name, subject
- Sort: Newest | Oldest | Priority | Last Updated
- Ticket preview cards:
- Subject (bold if unread)
- Student name + avatar
- Priority badge
- Status badge
- Last message preview (1 line)
- Timestamp
- Unread count indicator

Right Pane (Ticket Detail):

- **Header:** Subject + Status dropdown + Priority dropdown + Assignee dropdown + SLA timer
- **Student Info Card:** Name, Email, Enrolled courses, Account status
- **Message Thread:**
- Student messages: Left-aligned, white background, subtle border
- Admin messages: Right-aligned, --color-primary background, white text
- System messages: Centered, muted, italic (status changes)
- Attachments: Thumbnail + download link
- **Reply Composer:**
- RichTextEditor (compact)
- Attachment upload
- “Send & Resolve” / “Send & Keep Open” / “Send & Escalate” buttons
- Canned response selector (pre-written templates)
- **Sidebar:**
- Ticket metadata (Created, Source, Category)
- Related tickets
- Internal notes (admin-only)

6.11 Announcement Manager (/announcements)

List View:

- Table columns: Title | Target Audience | Priority | Status (Draft | Scheduled | Published | Expired) | Publish Date | Author | Actions
- Filter: Status, Date range, Author, Target
- Bulk actions: Delete, Duplicate

Create/Edit Form:

- Title* (text)
- Body* (RichTextEditor)
- Target Audience:
 - All Students
 - Specific Courses (multi-select)
 - Specific User Segments (e.g., “Inactive for 30 days”)
- Priority: Low | Normal | High | Critical
- Scheduling: Publish Now | Schedule (date/time picker) | Save as Draft
- Expiry: No expiry | Set date
- Preview: “Send Test” button (sends to admin email)
- **Publish Confirmation:** Shows estimated recipient count + “This will notify X students.”

6.12 User & Role Management (/users & /roles)

User List (/users):

- Table columns: Name | Email | Role | Status | Last Login | Created | Actions
- Filter: Role, Status, Date range
- “Add User” button → Modal with: Name, Email, Role, Permissions (if Moderator)

User Detail (/users/[id]):

- Profile info (read-only for Super Admin viewing Moderator)
- Activity log: All actions taken (courses published, tickets resolved, etc.)
- Permission matrix (if Moderator): Grid of features × permissions (View | Edit | Delete | Create)
- “Deactivate Account” / “Reset Password” / “Delete Account” (danger zone)

Role Configuration (/roles):

- Predefined roles: Super Admin (locked), Moderator (configurable)
- Permission categories:
 - Course Management (Create, Edit, Publish, Delete, Review Uploads)
 - Student Management (View, Edit, Suspend)

- Live Classes (Schedule, Edit, Cancel)
- Support (View Tickets, Respond, Resolve, Escalate)
- Reviews (View, Moderate, Delete)
- Service Requests (View, Update Status, Assign)
- Announcements (Create, Edit, Publish)
- Financial (View Revenue, View Transactions) — **Blocked for Moderators**
- Settings (View, Edit) — **Blocked for Moderators**
- User Management (View, Create, Edit, Delete) — **Blocked for Moderators for Super Admin accounts**

6.13 Settings (/settings/*)

General Settings (/settings/general):

- Platform Name
- Logo upload (light + dark variants)
- Favicon upload
- Contact Email / Phone / Address
- Social media links
- Default language & timezone
- Maintenance mode toggle

Payment Settings (/settings/payments):

- Gateway configuration (Stripe, Paystack, Flutterwave)
- API keys (masked, show/hide toggle)
- Webhook URL display
- Currency default
- Tax configuration
- Refund policy text

Notification Settings (/settings/notifications):

- Email provider (SendGrid, AWS SES)
- SMTP/API configuration
- Default sender name & email
- Notification templates list (click to edit)
- Push notification settings (future)

Security Settings (/settings/security):

- Password policy: Min length, complexity requirements, expiry

- Session timeout duration
- 2FA enforcement toggle
- Login attempt limits (max attempts, lockout duration)
- IP whitelist (optional)
- Audit log retention period

Integrations (/settings/integrations):

- Connected services grid: Google OAuth, Zoom, Jitsi, Cloudinary, Sentry, Analytics
- Each card: Service icon | Name | Status (Connected/Disconnected) | Configure button | Disconnect button
- Webhook management: List of registered webhooks with test/send buttons

6.14 Logs & Audit (/logs/*)

System Logs (/logs/system):

- Filter: Log level (ERROR | WARN | INFO | DEBUG), Date range, Source component
- Table: Timestamp | Level (colored badge) | Component | Message | Stack Trace (expandable)
- Auto-refresh toggle (30s interval)
- Export to JSON

Audit Trail (/logs/audit):

- Table: Timestamp | Admin | Action | Target Type | Target ID | Details | IP Address
- Filter: Admin, Action type, Date range, Target type
- Immutable records (no delete option)
- Export to CSV

Upload Logs (/logs/uploads):

- Table: Session ID | Course | Admin | Started | Completed | Status | Files Total | Files Success | Files Failed
- Expandable row: Per-file chunk status, error messages, retry history

7. State Management

7.1 Global State (Zustand)

```
// stores/authStore.ts
interface AdminAuthState {
  user: AdminUser | null;
  isAuthenticated: boolean;
  permissions: Permission[];
  hasPermission: (resource: string, action: string) => boolean;
  login: (credentials: LoginDTO) => Promise<void>;
  logout: () => void;
```

```

}

// stores/uiStore.ts
interface AdminUIState {
  sidebarCollapsed: boolean;
  sidebarPinned: boolean;
  theme: 'light' | 'dark' | 'system';
  toasts: AdminToast[];
  modals: Modal[];
  globalSearchOpen: boolean;
  toggleSidebar: () => void;
  openGlobalSearch: () => void;
}

// stores/courseDraftStore.ts
interface CourseDraftState {
  draft: Partial<Course> | null;
  currentStep: number;
  unsavedChanges: boolean;
  autoSave: () => Promise<void>;
  setStep: (step: number) => void;
  updateField: (field: string, value: any) => void;
}

// stores/uploadStore.ts
interface UploadStoreState {
  sessions: UploadSession[];
  activeSession: UploadSession | null;
  queue: UploadQueueItem[];
  isUploading: boolean;
  networkStatus: 'online' | 'offline' | 'slow';

  // Actions
  createSession: (courseId: string, files: File[]) => Promise<string>;
  addToQueue: (files: File[], lessonIds?: string[]) => void;
  startUpload: (sessionId: string) => void;
  pauseUpload: (fileId: string) => void;
  resumeUpload: (fileId: string) => void;
  retryFile: (fileId: string) => void;
  cancelFile: (fileId: string) => void;
  cancelAll: () => void;
  clearCompleted: () => void;
  assignLesson: (fileId: string, lessonId: string) => void;
  recoverSession: (sessionId: string) => Promise<void>;
  abandonSession: (sessionId: string) => void;

  // Chunk management
  updateChunkProgress: (fileId: string, chunkIndex: number, status: ChunkStatus) => void;
  markFileComplete: (fileId: string, fileUrl: string) => void;
  markFileFailed: (fileId: string, error: UploadError) => void;
}

```

```

// Network
setNetworkStatus: (status: 'online' | 'offline' | 'slow') => void;
handleNetworkChange: () => void;
}

interface UploadSession {
  id: string;
  courseId: string;
  courseName: string;
  status: 'preparing' | 'uploading' | 'paused' | 'completed' | 'failed';
  startedAt: Date;
  updatedAt: Date;
  expiresAt: Date;
  totalFiles: number;
  completedFiles: number;
  failedFiles: number;
  totalBytes: number;
  uploadedBytes: number;
  uploadToken: string;
  chunkSize: number;
  maxConcurrent: number;
}

interface UploadQueueItem {
  id: string;
  file: File;
  fileName: string;
  fileSize: number;
  fileType: string;
  lessonId: string | null;
  lessonName: string | null;
  status: 'waiting' | 'uploading' | 'paused' | 'completed' | 'failed' | 'cancelled' | 'processing';
  progress: number; // 0-100
  uploadedBytes: number;
  speed: number; // bytes per second
  eta: number; // seconds remaining
  chunks: Chunk[];
  retryCount: number;
  error: UploadError | null;
  checksum: string; // SHA-256
}

interface Chunk {
  index: number;
  start: number;
  end: number;
  size: number;
  status: 'pending' | 'uploading' | 'completed' | 'failed';
  retryCount: number;
  checksum: string;
}

```

```
interface UploadError {
  code: string;
  message: string;
  timestamp: Date;
  recoverable: boolean;
}
```

7.2 Server State (React Query)

```
// hooks/useCourses.ts
export const useCourses = (filters: CourseFilters, pagination: Pagination) =>
  useQuery({
    queryKey: ['admin', 'courses', filters, pagination],
    queryFn: () => fetchAdminCourses(filters, pagination),
    staleTime: 30 * 1000,
    keepPreviousData: true,
  });

// hooks/useUploadSession.ts
export const useUploadSession = (sessionId: string) =>
  useQuery({
    queryKey: ['upload', sessionId],
    queryFn: () => fetchUploadSession(sessionId),
    refetchInterval: 2000, // Poll every 2s for progress
  });

// hooks/useChunkedUpload.ts
export const useChunkedUpload = () => {
  const uploadStore = useUploadStore();

  const initiateUpload = async (courseId: string, files: File[]) => {
    // Calculate checksums for all files
    const fileChecksums = await Promise.all(
      files.map(f => calculateSHA256(f))
    );

    // Request upload session from backend
    const session = await api.post('/admin/uploads/initiate', {
      courseId,
      files: files.map((f, i) => ({
        name: f.name,
        size: f.size,
        type: f.type,
        checksum: fileChecksums[i],
      })),
    });

    uploadStore.createSession(session.data);
    return session.data.id;
  };
};
```

```

const uploadChunk = async (
  sessionId: string,
  fileId: string,
  chunk: Blob,
  chunkIndex: number,
  checksum: string
) => {
  const formData = new FormData();
  formData.append('chunk', chunk);
  formData.append('chunkIndex', chunkIndex.toString());
  formData.append('checksum', checksum);

  return api.put(`/admin/uploads/${sessionId}/chunks/${fileId}`, formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
    onUploadProgress: (progressEvent) => {
      uploadStore.updateChunkProgress(fileId, chunkIndex, {
        loaded: progressEvent.loaded,
        total: progressEvent.total,
      });
    },
  });
};

const processQueue = async (sessionId: string) => {
  const session = uploadStore.activeSession;
  if (!session) return;

  const queue = uploadStore.queue.filter(
    item => item.status === 'waiting' || item.status === 'failed'
  );

  // Process up to maxConcurrent files simultaneously
  const concurrencyLimit = session.maxConcurrent;
  const activeUploads = new Set<string>();

  for (const item of queue) {
    // Wait if we've hit the concurrency limit
    while (activeUploads.size >= concurrencyLimit) {
      await new Promise(resolve => setTimeout(resolve, 500));
    }

    activeUploads.add(item.id);

    uploadFile(sessionId, item).finally(() => {
      activeUploads.delete(item.id);
    });
  }
};

const uploadFile = async (sessionId: string, item: UploadQueueItem) => {

```

```

uploadStore.updateFileStatus(item.id, 'uploading');

const chunkSize = uploadStore.activeSession?.chunkSize || 5 * 1024 * 1024;
const totalChunks = Math.ceil(item.fileSize / chunkSize);

// Get already uploaded chunks from server (for resume)
const { data: uploadedChunks } = await api.get(
  `/admin/uploads/${sessionId}/files/${item.id}/chunks`
);

for (let i = 0; i < totalChunks; i++) {
  // Skip already uploaded chunks
  if (uploadedChunks.includes(i)) {
    uploadStore.updateChunkProgress(item.id, i, { status: 'completed', progress: 100 });
    continue;
  }

  const start = i * chunkSize;
  const end = Math.min(start + chunkSize, item.fileSize);
  const chunk = item.file.slice(start, end);
  const checksum = await calculateSHA256(chunk);

  let retries = 0;
  const maxRetries = 3;

  while (retries < maxRetries) {
    try {
      await uploadChunk(sessionId, item.id, chunk, i, checksum);
      uploadStore.updateChunkProgress(item.id, i, { status: 'completed', progress: 100 });
      break;
    } catch (error) {
      retries++;
      uploadStore.updateChunkProgress(item.id, i, {
        status: 'failed',
        retryCount: retries
      });

      if (retries >= maxRetries) {
        uploadStore.markFileFailed(item.id, {
          code: 'CHUNK_UPLOAD_FAILED',
          message: `Failed to upload chunk ${i} after ${maxRetries} attempts`,
          recoverable: true,
        });
        return;
      }

      // Exponential backoff: 1s, 2s, 4s
      await new Promise(resolve => setTimeout(resolve, 1000 * Math.pow(2, retries - 1)));
    }
  }
}

```

```

// All chunks uploaded — tell backend to assemble
try {
  const { data } = await api.post(`/admin/uploads/${sessionId}/files/${item.id}/complete`);
  uploadStore.markFileComplete(item.id, data.fileUrl);
} catch (error) {
  uploadStore.markFileFailed(item.id, {
    code: 'ASSEMBLY_FAILED',
    message: 'File assembly failed on server',
    recoverable: true,
  });
}
};

const recoverSession = async (sessionId: string) => {
  const { data: session } = await api.get(`/admin/uploads/session/${sessionId}`);
  uploadStore.recoverSession(session);

  // Resume any incomplete files
  const incompleteFiles = session.files.filter(
    (f: any) => f.status !== 'completed'
  );

  for (const file of incompleteFiles) {
    uploadStore.updateFileStatus(file.id, 'waiting');
  }

  // Auto-start if previously uploading
  if (session.status === 'uploading') {
    processQueue(sessionId);
  }
};

return {
  initiateUpload,
  uploadChunk,
  processQueue,
  uploadFile,
  recoverSession,
};
};

// hooks/useUploadRecovery.ts
export const useUploadRecovery = () => {
  const uploadStore = useUploadStore();

  useEffect(() => {
    // Check for interrupted sessions on mount
    const checkRecovery = async () => {
      const { data: sessions } = await api.get('/admin/uploads/sessions?status=interrupted');

```

```

    if (sessions.length > 0) {
      // Prompt admin to resume most recent session
      const mostRecent = sessions[0];
      uploadStore.setPendingRecovery(mostRecent);
    }
  };

  checkRecovery();

  // Handle browser beforeunload to warn about active uploads
  const handleBeforeUnload = (e: BeforeUnloadEvent) => {
    if (uploadStore.isUploading) {
      e.preventDefault();
      e.returnValue = 'You have active uploads. Are you sure you want to leave?';
    }
  };

  window.addEventListener('beforeunload', handleBeforeUnload);
  return () => window.removeEventListener('beforeunload', handleBeforeUnload);
}, []);

// Handle network status changes
useEffect(() => {
  const handleOnline = () => {
    uploadStore.setNetworkStatus('online');
    // Auto-resume paused uploads
    uploadStore.queue
      .filter(item => item.status === 'paused' || item.status === 'failed')
      .forEach(item => uploadStore.resumeUpload(item.id));
  };

  const handleOffline = () => {
    uploadStore.setNetworkStatus('offline');
    // Pause all active uploads
    uploadStore.queue
      .filter(item => item.status === 'uploading')
      .forEach(item => uploadStore.pauseUpload(item.id));
  };

  window.addEventListener('online', handleOnline);
  window.addEventListener('offline', handleOffline);

  return () => {
    window.removeEventListener('online', handleOnline);
    window.removeEventListener('offline', handleOffline);
  };
}, []);
};

// hooks/useDashboardMetrics.ts
export const useDashboardMetrics = (period: DateRange) =>

```

```

useQuery({
  queryKey: ['admin', 'dashboard', period],
  queryFn: () => fetchDashboardMetrics(period),
  staleTime: 60 * 1000,
});

// hooks/useActivityFeed.ts
export const useActivityFeed = (filters: ActivityFilters) =>
  useQuery({
    queryKey: ['admin', 'activity', filters],
    queryFn: () => fetchActivityFeed(filters),
    refetchInterval: 10000, // Real-time feel
  });

```

7.3 Local State

- Table sorting/filtering (React state)
- Form inputs (React Hook Form)
- Modal/drawer visibility
- Chart zoom/pan state
- Upload queue manager (complex local state for chunk tracking)

8. API Integration Patterns

8.1 Authentication & Authorization

1. Admin logs in → POST /api/admin/auth/login
2. Backend returns JWT + refresh token (httpOnly cookies)
3. Every request includes Bearer token via Axios interceptor
4. 403 Forbidden → Check permissions, show "Access Denied" page
5. 401 Unauthorized → Silent refresh → retry → if fails, redirect to login
6. Permission check on route entry: Redirect to Dashboard if no access

8.2 Chunked Upload Flow (Admin)

1. Admin selects files → Frontend calculates SHA-256 checksums
2. POST /api/admin/uploads/initiate
→ Response: { sessionId, chunkSize: 5242880, maxConcurrent: 5 }
3. For each file:
 - a. Split into 5MB chunks
 - b. Upload chunks: PUT /api/admin/uploads/chunk
Headers: X-Upload-Session, X-Chunk-Index, X-Chunk-Checksum
 - c. Parallel execution: Up to 5 concurrent chunks across all files
 - d. Per-chunk retry: 3 attempts with exponential backoff (1s, 2s, 4s)
4. POST /api/admin/uploads/complete/{sessionId}
→ Backend validates all chunks, assembles file, verifies checksum
→ Returns: { fileUrl, duration, thumbnailUrl }
5. Frontend associates fileUrl with lesson record
6. On browser crash: GET /api/admin/uploads/session/{sessionId}

→ Returns uploaded chunks list → Resume from missing chunks

8.3 Real-Time Updates

- **Activity Feed:** Server-Sent Events (SSE) on `/api/admin/events/stream`
- **Upload Progress:** Polling (2s) + Optimistic UI updates
- **Notification Badges:** Polling (30s) + SSE push for critical alerts
- **Dashboard Metrics:** Polling (60s) with manual refresh button

9. Responsive Breakpoints

Name	Width	Layout Changes
sm	< 640px	Single column, stacked cards, hamburger nav, bottom sheets
md	640px –1023px	2-column grids, collapsible sidebar (icon-only or overlay)
lg	1024px –1279px	Full sidebar, 3-column metric cards, split-pane tickets
x1	1280px –1535px	4-column metric cards, expanded data tables
2x1	1536px	Maximum content width (1440px centered), spacious layouts

Admin-Specific Responsive Behaviors

- **DataTables:** Horizontal scroll with sticky first column (student name/course title)
- **Charts:** Stacked vertically on mobile, side-by-side on desktop
- **Forms:** Single column on mobile, 2-column grid on desktop
- **Modals:** Full-screen on mobile, centered on desktop
- **Split Panes:** Collapsible drawers on tablet, persistent panes on desktop

10. Accessibility Requirements

10.1 Keyboard Navigation

- Full sidebar navigation via arrow keys + Enter
- Table: Arrow keys navigate cells, Enter opens detail, Space selects row
- Modal: Tab traps focus, Escape closes
- Form wizard: Step navigation via keyboard
- Global shortcuts:
 - `Cmd/Ctrl + K`: Global search
 - `Cmd/Ctrl + /`: Keyboard shortcuts help modal
 - `Cmd/Ctrl + S`: Save form (where applicable)
 - `Esc`: Close modals, drawers, dropdowns

10.2 Screen Reader Support

- ARIA labels for all icon-only buttons
- `aria-live="polite"` for activity feed and toast notifications
- Table headers with `scope="col"`
- Sort indicators with `aria-sort`
- Progress bars with `aria-valuenow`, `aria-valuemin`, `aria-valuemax`
- Skip links for main content

10.3 Visual Accessibility

- High contrast mode support (respect `prefers-contrast: high`)
 - Reduced motion (disable chart animations, table transitions)
 - Focus rings: 2px solid `--color-accent`, offset 2px
 - Status not conveyed by color alone (icons + text in badges)
-

11. Performance Considerations

11.1 Data Handling

- **Virtualized Tables:** React Table + TanStack Virtual for lists > 100 rows
- **Pagination:** Server-side for all list endpoints (default 25/page)
- **Debounced Search:** 300ms delay on filter inputs
- **Optimistic Updates:** Status changes (ticket resolved, course published) update UI immediately, rollback on error

11.2 Chart Performance

- **Data Downsampling:** For time-series charts with > 1000 points
- **Lazy Loading:** Charts below fold load when scrolled into view
- **Canvas Rendering:** For complex charts (future optimization)

11.3 Bundle Optimization

- **Route Splitting:** Each major module is a separate chunk
 - **Dynamic Imports:** Heavy components (charts, rich text editor, video player) loaded on demand
 - **Tree Shaking:** Recharts components imported individually
-

12. Error Handling & Edge Cases

12.1 Network Errors

- **Retry Strategy:** Auto-retry idempotent GET requests (3 attempts)
- **Offline Indicator:** Banner when connection lost, queue non-critical mutations
- **Timeout Handling:** 10s timeout with “Request timed out” message + retry

12.2 Permission Errors

- **Route Guards:** Redirect to Dashboard if user lacks permission

- **UI Hiding:** Buttons/links hidden based on permissions (not just disabled)
- **Action Blocking:** Backend enforces all permissions; frontend is cosmetic

12.3 Data Conflicts

- **Concurrent Editing:** Optimistic locking with version numbers; conflict warning if another admin edited the same record
- **Stale Data:** “This data has been updated by another user. Refresh?” prompt

12.4 Upload Failures & Recovery

Pre-Upload Validation (Client-Side):

- **File Type Whitelist:** MP4, MOV, AVI, MKV, WEBM (video); PDF, DOCX, PPTX, XLSX (resources)
- **File Size Limits:** Video max 10GB; Resource max 100MB
- **Video Duration:** Minimum 30 seconds (warn); maximum 4 hours (hard limit)
- **Resolution Check:** Minimum 720p recommended (warn if below)
- **Duplicate Detection:** Compare filename + size against existing course files
- **Total Course Size:** Warn if course exceeds 50GB aggregate

Upload Failure Scenarios & Recovery:

Failure Type	Detection	Recovery Strategy	User Feedback
Network Interruption	XMLHttpRequest onerror / timeout	Auto-pause all uploads; queue retry on reconnect; resume from last successful chunk	“Connection lost. Uploads paused. Will resume automatically.”
Chunk Upload Timeout	Request exceeds 30s	Retry chunk (max 3 attempts, exponential backoff: 1s, 2s, 4s)	“Retrying chunk X... (attempt Y/3)”
Chunk Checksum Mismatch	Backend rejects chunk	Re-read chunk from File API, recalculate checksum, retry	“Verifying file integrity...”
Server Storage Full	507 Insufficient Storage	Pause all uploads; notify admin to free space or upgrade	“Server storage full. Contact Super Admin.”
Invalid File Format	Backend validation (magic numbers)	Reject file immediately; allow replacement	“Invalid format. Please upload MP4, MOV, or MKV.”
Session Expired	401/403 on chunk upload	Prompt to re-authenticate; preserve queue state	“Session expired. Please log in again to resume.”

Table 8 – continued

Failure Type	Detection	Recovery Strategy	User Feedback
Browser Crash / Close	beforeunload handler + session persistence	On return, detect interrupted session via <code>GET /api/admin/uploads/sessions?status=interrupted;</code> prompt to resume	“You have unfinished uploads. Resume where you left off?”
Concurrent Upload Limit	Backend returns 429 Too Many Requests	Reduce <code>maxConcurrent</code> dynamically; queue excess files	“Slowing uploads to maintain stability...”
File Corruption (post-assembly)	Backend checksum mismatch after assembly	Re-queue entire file for re-upload	“File verification failed. Re-uploading...”

Session Persistence Strategy:

- Upload session ID stored in `localStorage` with timestamp
- Queue state serialized to `localStorage` every 5 seconds during active uploads
- On page load: Check `localStorage` for active sessions → Query backend for status → Rehydrate queue
- Session TTL: 7 days (backend); after expiry, files deleted, session abandoned

Atomic Publishing Guard:

- Course cannot transition to “Published” if any upload session is active or failed
- Validation checklist explicitly checks: `allUploadsCompleted && allFilesVerified`
- If publish attempted with incomplete uploads: Block with error list + “Complete uploads first” CTA

13. Third-Party Integrations

Service	Purpose	Integration Point
Recharts	Data visualization	Revenue charts, analytics dashboards
React Table / TanStack Table	Advanced data tables	All list views (courses, students, transactions)
React Hook Form + Zod	Form validation	All forms (course creation, settings, user management)
React DnD / @dnd-kit	Drag-and-drop	Course module/lesson reordering, Kanban board
Date-fns	Date manipulation	All date displays, scheduling, filtering
Zustand	Global state	Auth, UI preferences, course draft

Table 9 – continued

Service	Purpose	Integration Point
TanStack Query	Server state	All API data fetching, caching, synchronization
Axios	HTTP client	API communication with interceptors
Sentry	Error tracking	Global error boundary, performance monitoring
FullCalendar / React-Big-Calendar	Calendar views	Live class scheduling

14. File Structure

```

grammarmcetamol-admin-interface/
├─ app/                                # Next.js App Router
│   ├─ (auth)/
│   │   ├─ login/page.tsx
│   │   └─ forgot-password/page.tsx
│   └─ (dashboard)/
│       ├─ page.tsx                    # Dashboard
│       └─ courses/
│           ├─ page.tsx                # Course List
│           ├─ create/page.tsx        # Course Creation Wizard
│           └─ [id]/
│               ├─ page.tsx            # Course Detail
│               ├─ upload/page.tsx
│               ├─ analytics/page.tsx
│               └─ versions/page.tsx
│       └─ live-classes/
│           ├─ page.tsx
│           ├─ create/page.tsx
│           └─ [id]/page.tsx
│       └─ students/
│           ├─ page.tsx
│           └─ [id]/page.tsx
│       └─ service-requests/
│           ├─ page.tsx
│           └─ [id]/page.tsx
│       └─ reviews/page.tsx
│       └─ revenue/
│           ├─ page.tsx
│           └─ transactions/page.tsx
│       └─ analytics/
│           ├─ overview/page.tsx
│           ├─ students/page.tsx
│           ├─ courses/page.tsx
│           └─ revenue/page.tsx

```

```

| | | └─ announcements/
| | | | └─ page.tsx
| | | | └─ create/page.tsx
| | | └─ support/
| | | | └─ tickets/page.tsx
| | | | └─ tickets/[id]/page.tsx
| | | └─ users/
| | | | └─ page.tsx
| | | | └─ create/page.tsx
| | | | └─ [id]/page.tsx
| | | └─ roles/page.tsx
| | | └─ settings/
| | | | └─ general/page.tsx
| | | | └─ payments/page.tsx
| | | | └─ notifications/page.tsx
| | | | └─ security/page.tsx
| | | | └─ integrations/page.tsx
| | | | └─ backup/page.tsx
| | └─ logs/
| | | └─ system/page.tsx
| | | └─ audit/page.tsx
| | | └─ uploads/page.tsx
| └─ layout.tsx # Root layout (sidebar, header, providers)
| └─ globals.css
| └─ loading.tsx
|
└─ components/
  └─ layout/
    | └─ Sidebar.tsx
    | └─ TopHeader.tsx
    | └─ Breadcrumb.tsx
    | └─ PageHeader.tsx
    | └─ MainContent.tsx
  └─ ui/ # Primitive components (shared with student app)
    | └─ Button.tsx
    | └─ Input.tsx
    | └─ Modal.tsx
    | └─ Toast.tsx
    | └─ Badge.tsx
    | └─ Dropdown.tsx
    | └─ Tabs.tsx
  └─ data-display/
    | └─ DataTable.tsx
    | └─ StatCard.tsx
    | └─ StatusBadge.tsx
    | └─ ProgressBar.tsx
    | └─ UploadQueueItem.tsx
    | └─ ActivityFeed.tsx
    | └─ EmptyState.tsx
  └─ charts/
    | └─ LineChart.tsx

```

```

|   |   ├── BarChart.tsx
|   |   ├── PieChart.tsx
|   |   └── Sparkline.tsx
|   ├── forms/
|   |   ├── FormSection.tsx
|   |   ├── RichTextEditor.tsx
|   |   ├── ImageUploader.tsx
|   |   ├── VideoUploader.tsx
|   |   ├── DateTimePicker.tsx
|   |   └── MultiSelect.tsx
|   ├── course/
|   |   ├── CourseWizard.tsx
|   |   ├── ModuleManager.tsx
|   |   ├── LessonList.tsx
|   |   ├── UploadManager.tsx
|   |   └── CoursePreview.tsx
|   ├── support/
|   |   ├── TicketList.tsx
|   |   ├── TicketDetail.tsx
|   |   ├── MessageThread.tsx
|   |   └── ReplyComposer.tsx
|   └── kanban/
|       ├── KanbanBoard.tsx
|       ├── KanbanColumn.tsx
|       └── KanbanCard.tsx
|
|   ├── hooks/
|   |   ├── useAuth.ts
|   |   ├── usePermissions.ts
|   |   ├── useCourses.ts
|   |   ├── useStudents.ts
|   |   ├── useUploadSession.ts
|   |   ├── useDashboardMetrics.ts
|   |   ├── useActivityFeed.ts
|   |   ├── useDebounce.ts
|   |   └── useMediaQuery.ts
|
|   ├── stores/
|   |   ├── authStore.ts
|   |   ├── uiStore.ts
|   |   ├── courseDraftStore.ts
|   |   ├── uploadStore.ts
|   |   └── notificationStore.ts
|
|   ├── lib/
|   |   ├── api.ts                # Axios instance + interceptors
|   |   ├── utils.ts
|   |   ├── constants.ts
|   |   └── types/
|   |       ├── admin.ts
|   |       └── course.ts

```

```
|   | student.ts
|   | payment.ts
|   | support.ts
|
| public/
|   | images/
|   | icons/
|
| middleware.ts           # Route protection, role checks
| next.config.js
| tailwind.config.ts
| tsconfig.json
| package.json
```

15. Security Considerations

- **Role-Based UI:** Components conditionally render based on permissions; backend is the ultimate authority
- **Input Sanitization:** All rich text sanitized via DOMPurify before display
- **CSRF Protection:** SameSite cookies, CSRF tokens for mutations
- **Audit Logging:** Every admin action logged with timestamp, admin ID, IP, action details
- **Session Management:** Automatic timeout after inactivity, concurrent session limits
- **Data Masking:** Sensitive fields (API keys, payment tokens) masked by default, reveal on click
- **File Upload Security:**
 - Virus scanning via ClamAV or cloud scanning API before processing
 - File type validation via magic numbers (not just extension)
 - Size limits enforced client-side and server-side
 - Quarantine zone: Uploaded files scanned before moving to public CDN
 - Admin upload restrictions: Only authenticated admins with `course:create` or `course:edit` permissions
 - Upload rate limiting: Max 5 concurrent uploads per admin, max 50GB per hour
 - Signed upload URLs with short expiry (15 minutes) for direct-to-S3 uploads (future optimization)

End of Document